

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE & MANAGEMENT

Project – Low Level Design on AI Driven Self-Healing Cloud System for Finance

Student Name:

NANDITHA KRISHNA EM

23BCAICD089

LH-06

Industry Mentor:

Guided by:

Table of Contents

1. Introduction.....	3
1.1. Scope of the document.....	3
1.2. Intended Audience.....	3
1.3. System overview.....	3
2. System Design.....	4
2.1. Application Design.....	4
2.2. Process Flow.....	4
2.3. Information Flow.....	5
2.4. Components Design.....	5
2.5. Key Design Considerations.....	7
2.6. API Catalogue.....	7
3. Data Design.....	7
3.1. Data Model.....	7
3.2. Data Access Mechanism.....	8
3.3. Data Retention Policies.....	8
3.4. Data Migration.....	8
4. Interfaces.....	8
5. State and Session Management.....	9
6. Caching.....	9
7. Non-Functional Requirements.....	9
7.1. Security Aspects.....	9
7.2. Performance Aspects.....	9
8. References.....	9

1. Introduction

1.1 Scope of the Document

This document provides a detailed internal design of the AI-based self-healing system. It explains how data is generated, processed, analyzed, and modified through automated corrective mechanisms.

The scope includes:

- Internal module behavior
- Data processing logic
- Machine learning implementation
- Self-healing strategy

The document focuses on implementation-level clarity while maintaining modular design principles.

1.2 Intended Audience

This document is intended for:

- Developers who want to understand or extend the system
- Academic reviewers evaluating the technical depth
- Learners exploring anomaly detection and system automation
- It assumes basic knowledge of Python, machine learning, and data processing.

1.3 System Overview

The system is designed to simulate a real-world monitoring environment where system performance and financial transactions are continuously analyzed.

It uses an unsupervised machine learning algorithm (Isolation Forest) to identify abnormal behavior in system metrics such as CPU usage, latency, and error rates.

Once anomalies are detected, the system automatically triggers corrective actions. These actions modify the system state, effectively simulating recovery. The updated data is stored and visualized to demonstrate the impact of self-healing.

2. System Design

2.1 Application Design

The application is structured into independent modules to ensure separation of concerns and maintainability.

- | | | |
|---|-------------------|--------------|
| • Data | Generation | Layer |
| Responsible for creating realistic datasets with both normal and anomalous behavior. | | |
| • Processing | | Layer |
| Handles anomaly detection and applies machine learning models along with feature engineering. | | |
| • Self-Healing | | Layer |
| Applies rule-based corrective actions based on anomaly severity and system thresholds. | | |
| • Presentation | | Layer |
| Provides visualization and monitoring through a Streamlit dashboard. | | |

This layered design ensures flexibility and scalability for future enhancements.

2.2 Process Flow

The system operates in a sequential pipeline:

1. Synthetic data is generated with time-based variations and injected anomalies
2. Data is stored locally in CSV format
3. The anomaly detection module processes the data using scaled features
4. Anomalies are identified and labeled
5. Severity levels are assigned based on predefined thresholds
6. The self-healing module evaluates anomaly conditions
7. Corrective actions are applied to affected records
8. Updated (healed) data is stored separately
9. The dashboard visualizes both original and healed system behavior
10. Task Scheduler automatically triggers system execution at regular intervals

This pipeline ensures a complete lifecycle from detection to recovery.

2.3 Information Flow

The system follows a structured data flow:

- Data Generator → CSV Storage
- CSV → DataFrame (Pandas)
- DataFrame → Anomaly Detection Model
- Detection Output → Self-Healing Module
- Modified Data → Healed Dataset
- Both datasets → Dashboard Visualization

This flow ensures clear separation between raw, processed, and healed data.

2.4 Components Design

1. Data Generator Module

- Generates time-series data using timestamps
- Applies load variations based on time of day
- Injects anomalies randomly (approx. 5%)
- Simulates real-world conditions such as:
 - sudden spikes
 - system failures
 - attack scenarios

This ensures the dataset is realistic and suitable for anomaly detection.

2. Anomaly Detection Module

- Uses Isolation Forest (unsupervised ML algorithm)
- Applies feature scaling using StandardScaler
- Performs feature engineering to improve detection accuracy:
 - CPU to latency ratio
 - Error rate to CPU ratio
 - Log transformation for transaction amount

The model identifies outliers based on deviation from normal patterns. Severity classification is applied based on threshold rules to categorize anomalies into LOW, MEDIUM, and HIGH.

3. Self-Healing Module

This is the core of the system.

- Identifies critical anomalies using maximum values

- Applies corrective transformations:
 - CPU load reduction
 - Latency optimization
 - Error rate normalization

Example logic:

- If CPU usage exceeds threshold → reduce load
- If latency is high → simulate service restart
- If error rate spikes → trigger recovery mechanism

The system modifies the dataset to reflect a stabilized state, simulating real system recovery.

4. Task Scheduler Module

- Uses Windows Task Scheduler for automation
- Executes main.py at fixed intervals (e.g., every 5 minutes)
- Ensures continuous data generation, anomaly detection, and self-healing
- Eliminates need for manual execution

Execution Flow:

- Scheduler triggers Python script
- Script processes data and updates output files
- Dashboard reflects updated system state

5. Dashboard Module

- Built using Streamlit for simplicity and interactivity
- Uses Plotly for dynamic visualizations
- Displays:
 - system metrics over time
 - anomaly detection results
 - severity distribution
 - before vs after healing comparison

This allows users to clearly understand system behavior and recovery impact.

2.5 Key Design Considerations

- Maintain modular architecture for easy updates
- Ensure realistic simulation of system behavior
- Use efficient algorithms for anomaly detection
- Provide clear visualization for better interpretation
- Allow extensibility for real-time systems

2.6 API Catalogue

Currently, the system does not integrate external APIs.

However, internal function interfaces act as modular APIs:

- anomaly detection functions
- data generation functions
- self-healing functions

Future integration can include REST APIs for real-time monitoring.

3. Data Design

3.1 Data Model

The system uses structured tabular data.

System Metrics:

Includes performance indicators such as CPU, memory, latency, and error rates. These attributes are interdependent and simulate real system behavior.

Transaction Data:

Includes performance indicators such as CPU, memory, latency, and error rates. These attributes are interdependent and simulate real system behavior.

3.2 Data Access Mechanism

- Data is stored in CSV format
- Accessed using Pandas DataFrames
- Processed in-memory for fast computation
- Saved back after transformation

3.3 Data Retention Policies

- Logs stored for tracking system actions
- Suspicious transactions stored separately
- Healed data maintained for comparison

This ensures traceability and audit capability

3.4 Data Migration

Currently not implemented.

Future enhancements may include:

- migration to relational databases
- cloud-based storage solutions

4. Interfaces

The system provides:

- Command-line interface for execution
- Web-based dashboard for visualization

Users can:

- monitor system health
- observe anomalies
- view recovery actions

5. State and Session Management

- Backend operates in a stateless manner
- Each run processes fresh or existing data independently
- Streamlit manages session state for UI

6. Caching

- Streamlit caching is used to optimize performance
- Prevents redundant data loading
- Enhances responsiveness of dashboard

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data is processed
- Local execution ensures controlled environment
- Future enhancements may include authentication and encryption

7.2 Performance Aspects

- Efficient ML model suitable for medium-scale data
- Fast processing using Pandas
- Lightweight UI rendering
- Automated execution ensures timely anomaly detection and reduces system downtime

8. References

- Scikit-learn Documentation
- Streamlit Documentation
- Pandas Documentation



Innovation Centre for Education

